

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»

Лабораторная работа №1
по курсу «Программирование графических процессоров и технологии
параллельного программирования»

Освоение программного обеспечения для работы с технологией
CUDA. Примитивные операции над векторами

Выполнил: Ефимов Сергей Юрьевич
Группа: М80-403Б-23
Преподаватель: А.Ю. Морозов

Москва, 2026

Условие

Ознакомление и установка программного обеспечения для работы с программно-аппаратной архитектурой параллельных вычислений(CUDA). Реализация одной из примитивных операций над векторами. В качестве вещественного типа данных необходимо использовать тип данных double. Все результаты выводить с относительной точностью 10^{-10} . Ограничение: $n < 2^{25}$.

Вариант 5.

Входные данные. На первой строке задано число n -- размер векторов. В следующих 2-х строках, записано по n вещественных чисел -- элементы векторов.

Выходные данные. Необходимо вывести n чисел -- результат поэлементного нахождения максимума исходных векторов.

Программное и аппаратное обеспечение

Видеокарта

```
+-----+
| NVIDIA-SMI 615.71.09           KMD Version: 615.71.09    CUDA UMD Version: 13.4
|
+-----+-----+-----+
| GPU Name          Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf    Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|                                   |        MIG M.     |                    |
+=====+=====+=====+
| 0 NVIDIA GeForce GTX 1660 ...  Off| 00000000:01:00.0 On |          N/A |
| 43%  31C   P8      11W / 125W | 557MiB / 6144MiB | 16%      Default |
|                                   |        N/A       |                    |
+-----+-----+-----+
```

Device 0

Compute capability : 7.5

Name : NVIDIA GeForce GTX 1660 SUPER

Total Global Memory : 6034096128

Shared memory per block : 49152

Registers per block : 65536

Warp size : 32

Max threads per block : (1024, 1024, 64)

Max block : (2147483647, 65535, 65535)

Total constant memory : 65536

Multiprocessors count : 22

Процессор

Архитектура: x86_64

CPU op-mode(s): 32-bit, 64-bit

Address sizes: 39 bits physical, 48 bits virtual

Порядок байт: Little Endian
CPU(s): 12
On-line CPU(s) list: 0-11
ID производителя: GenuineIntel
Имя модели: 11th Gen Intel(R) Core(TM) i5-11500 @ 2.70GHz
Семейство ЦПУ: 6
Модель: 167
Thread(s) per core: 2
Ядер на сокет: 6
Сокетов: 1
Степпинг: 1
Microcode version: 0x66
CPU(s) scaling MHz: 92%
CPU max MHz: 4600,0000
CPU min MHz: 800,0000
BogoMIPS: 5424,00

Оперативная память

	total	used	free	shared	buff/cache	available
Mem:	15Gi	7,0Gi	1,4Gi	237Mi	7,8Gi	8,5Gi
Swap:	15Gi	6,2Mi	15Gi			

Жесткий диск

NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPOINTS
sda	8:0	0	931,5G	0	disk	
└─sda1	8:1	0	931G	0	part	
sr0	11:0	1	1024M	0	rom	
zram0	253:0	0	15,4G	0	disk [SWAP]	
nvme0n1	259:0	0	1,9T	0	disk	
└─nvme0n1p1	259:1	0	4G	0	part	/boot
└─nvme0n1p2	259:2	0	1,9T	0	part	/var/tmp
						/var/log
						/var/cache
						/home
						/root
						/srv
						/

Система

Linux cachyos-x8664 7.2.6-1-cachyos #1 SMP PREEMPT_DYNAMIC Thu, 17 Sep 2026
15:20:14 +0000 x86_64 GNU/Linux

IDE

CLion от JetBrains

Компилятор:

Для локального тестирования использовался CMake со следующей конфигурацией:

```
cmake_minimum_required(VERSION 4.3)
set(CMAKE_CUDA_COMPILER /opt/cuda/bin/nvcc)
set(CUDAToolkit_ROOT /opt/cuda)
project(pgp_tpp LANGUAGES CXX CUDA)
set(CMAKE_CXX_STANDARD 14)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CUDA_STANDARD 14)
set(CMAKE_CUDA_STANDARD_REQUIRED ON)
```

При отправке использовался makefile

```
NVCC = /usr/bin/nvcc
CFLAGS = --std=c++14
SOURCES = lab1.cu
BIN = lab1
all:
    $(NVCC) $(CFLAGS) -o $(BIN) $(SOURCES)
```

Метод решения

Для реализации алгоритма используя CUDA нужна функция с `__global__` для запуска подсчета на GPU. В функции происходит реализация поэлементного нахождения максимума исходных векторов с распределением потоков. В главной функции `main` происходит считывание данных, выделение памяти на GPU и перенос данных из CPU в GPU. Затем запускается функция для подсчета задания. Полученный результат переносится из GPU в CPU. После этого программа выводит результат и освобождает память на GPU.

Описание программы

Функция `maxKernel` необходима для поэлементного нахождения максимума исходных векторов используя CUDA. Внутри используется `blockIdx` (номер блока), `blockDim` (кол-во потоков внутри блока), `threadIdx` (номер потока) для присвоения уникального `idx` каждому потоку, а также `blockDim` и `gridDim` (кол-во блоков внутри сетки) для случая, если `n >` кол-во потоков (в данном случае некоторые потоки могут взять на себя выполнение нескольких итераций). В главной функции `main` происходит считывание входных данных, затем с помощью `cudaMalloc` выделяется память на GPU под входные вектора и результат. С помощью `cudaMemcpy` значения векторов из CPU записываются в ранее выделенные блоки в GPU. После берем количество потоков 256 и в зависимости от размера `n` выделяем необходимое количество блоков. Далее запускаем выполнение функции `maxKernel`, ждем выполнение всех потоков с помощью `cudaDeviceSynchronize()`. Снова используем `cudaMemcpy`, но на этот раз для извлечения записи из CUDA в CPU и выводим результат. В конце работы освободим память.

Результаты

Время работы ядер с различными конфигурациями

n = 1000

Time: milliseconds (ms)

Blocks	Threads	Time (ms)
1	32	0.021178
1	64	0.011530
1	128	0.008739
1	256	0.007194
1	512	0.005763
1	1024	0.005830
2	32	0.011194
2	64	0.008397
2	128	0.006957
2	256	0.005341
2	512	0.005165
2	1024	0.005661
4	32	0.008435
4	64	0.006976
4	128	0.005338
4	256	0.004890
4	512	0.005251
4	1024	0.005770
8	32	0.006835
8	64	0.005350
8	128	0.005117
8	256	0.005226
8	512	0.005203
8	1024	0.005651
16	32	0.005306
16	64	0.005181
16	128	0.004950
16	256	0.004970
16	512	0.005424
16	1024	0.005728
32	32	0.005107
32	64	0.005053
32	128	0.005434
32	256	0.005712
32	512	0.005126
32	1024	0.005997

64	32	0.004954
64	64	0.005219
64	128	0.006720
64	256	0.005888
64	512	0.005754
64	1024	0.005926
128	32	0.005104
128	64	0.004944
128	128	0.005325
128	256	0.005226
128	512	0.005846
128	1024	0.006336
256	32	0.005030
256	64	0.005366
256	128	0.004918
256	256	0.005258
256	512	0.006128
256	1024	0.008346
512	32	0.006115
512	64	0.005802
512	128	0.005859
512	256	0.005997
512	512	0.007885
512	1024	0.012400
1024	32	0.007133
1024	64	0.006774
1024	128	0.006624
1024	256	0.008019
1024	512	0.011446
1024	1024	0.020634

Сравнение гри и сри

n = 3

CPU time: 0.001039 ms

GPU kernel time: 0.008384 ms

Blocks: 1

Threads: 256

Results: OK

n = 100000

CPU time: 1.126 ms

GPU time: 0.092224 ms

Blocks: 391

Threads: 256

Results: OK

Выводы

Данная лабораторная работа познакомила нас с CUDA и его базовым использованием в C++. Данный код отлично справляется с задачей ознакомления и возможностью сравнить разницу между `cpu` и `gpu`. Реализация рабочего кода не составила особых сложностей. При решении задачи были проблемы только с составлением `makefile` и ошибкам по невнимательности. По сравнению видно, что самый оптимальный результат был в случае, когда кол-во потоков было равно n , то есть каждый поток выполнил одну итерацию. В сравнении CPU и GPU видно, что на малых числах CPU работает быстрее, однако при более больших числах GPU начинает работать лучше